

Impact of Secure Aggregation and Differential Privacy on Federated Network Intrusion Detection Systems

Carlos Andrés Peña Arias
Department of Systems and Computing Engineering
Universidad de los Andes
Bogotá, Colombia
c.penaa@uniandes.edu.co

Abstract—Federated Learning (FL) facilitates the joint training of machine learning models without centralizing sensitive data, making it highly suitable for Network Intrusion Detection Systems (NIDS). However, FL is vulnerable to inference attacks that may compromise training data privacy and data exposure attacks by honest-but-curious servers [1]. To mitigate these risks, privacy-preserving mechanisms such as Differential Privacy (DP) and Secure Aggregation (SA) are employed [2]. This work presents an empirical evaluation of the impact of DP and SA on the performance of a federated NIDS using industry-level framework NVIDIA FLARE (NVFlare).

Index Terms—Federated Learning, Network Intrusion Detection Systems, Differential Privacy, Secure Aggregation, NVFlare, Cybersecurity

I. INTRODUCTION

Modern NIDS rely more and more on machine learning (ML) and deep learning (DL), which require access to large datasets to efficiently and accurately identify cyber threats [2]. Traditional centralized approaches introduce privacy risks and regulatory challenges due to data concentration and storage of sensitive information.

FL addresses these issues by enabling distributed model training without sharing raw data [3]. However, FL remains vulnerable to inference attacks, including membership inference, which can expose whether specific samples were used during training [1]. Additionally, FL is vulnerable to data exposure attacks and data poisoning attacks by honest-but-curious servers [4].

To mitigate these risks, additional privacy-preserving mechanisms such as DP and SA are integrated into FL pipelines [2]. DP protects individual data points by injecting controlled noise, while SA ensures that the server only observes aggregated model updates.

This paper evaluates the impact of these mechanisms on model performance, privacy, and computational cost in a realistic federated NIDS setting using industry-standard framework NVFlare.

II. RELATED WORK

Federated Learning has gained significant attention across domains such as critical infrastructure, healthcare, and finance

due to its privacy-preserving capabilities [3]. In the context of NIDS, FL enables collaborative training across distributed environments without exposing sensitive data.

Despite these advantages, FL introduces several challenges, including vulnerability to inference attacks, communication overhead, and data heterogeneity [2]. In particular, inference attacks pose a major risk, as adversaries can extract sensitive information from model updates [1].

DP has been widely adopted as a defense mechanism against such attacks, ensuring that individual data contributions cannot be easily inferred [5]. However, the addition of noise typically degrades model performance.

SA, on the other hand, prevents the central server from accessing individual client updates by using cryptographic techniques such as homomorphic encryption [2]. While this method preserves accuracy, it introduces additional computational and communication overhead.

Recent work has explored privacy-preserving FL for intrusion detection, but often lacks comprehensive experimental evaluation using large-scale datasets and industry frameworks [6]. Additionally, there is limited analysis of the quantitative trade-off between privacy, performance and computational cost in this context.

III. METHODOLOGY

A. Experimental Setup

All experiments were conducted on a single NVIDIA RTX 4090 GPU with 24GB of memory, a Ryzen 5800x3D CPU and 32GB of RAM. This setup reflects a high end consumer-grade machine, which is more accessible than enterprise-grade hardware while still providing sufficient computational power for training complex models.

For the experiments, NVFlare framework was used to implement the federated learning pipeline. The framework was run in a WSL environment on the same machine, simulating multiple clients for the federated setup. The same codebase was used for both the centralized and federated experiments to ensure a fair comparison. The code is available in the following repository: <https://github.com/carandp/FL-NIDS>.

B. Dataset

The NF-CSE-CIC-IDS2018-v3 dataset was used in the experiments. The dataset is derived from NF-CSE-CIC-IDS2018, a dataset collected from a large scale infrastructure deployed in AWS, containing 50 attacker machines and 5 departments composed of 420 PCs and 30 servers [7] [8]. This dataset, provides aggregated representations of network traffic suitable for large-scale intrusion detection [9]. Due to hardware constraints, a representative subset of the main dataset was used for training and evaluation.

To achieve a representative sample, there was a study done to analyze the main ports in the dataset, to then be grouped as follows:

Category	Port	Flows	Distribution
App	80 (HTTP)	6,045,943	Benign: 71.1% Attack: 28.9%
	8080 (HTTP)		
	443 (HTTPS)		
File Server	21 (FTP)	5,578,635	Benign: 87.46% Attack: 12.54%
	22 (SFTP)		
	138 (NetBIOS)		
	139 (NetBIOS)		
	445 (SMB)		
	3389 (RDP)		
AD and Others	53 (DNS)	7,385,573	Benign: 98.54% Attack: 1.46%
	135 (RPC)		
	500 (VPN)		
	1433 (MSSQL)		
	3306 (MySQL)		
	5060 (VoIP)		

TABLE I: Port Categories with Flow Distribution

As shown in Table I, it can be observed that the AD and Others Category has a very high class imbalance, with only 1.46% of the flows being attacks. To mitigate this imbalance, all the attack samples from this category were included in the training set, while only a representative sample of the benign flows were included. For the other two categories, a representative sample of both benign and attack flows were included in the training set. To be specific, for each category and class, the samples were sorted by their start time and the first 25% of the samples were included in the training set, while the rest were discarded. This approach ensures that the training set is representative of the overall distribution while also addressing class imbalance in the AD and Others category. The resulting training subset then contains:

As shown in Table II, the final training subset contains still contains a significant number of samples, while being more manageable for training and evaluation. This subset is used for all subsequent experiments in this work.

The decision behind the category strategy is due to real life constraints. Where there is no prior knowledge of the attack distribution, and the training data can't be grouped by attack type, but it can be grouped by port and related services.

Category	Port	Flows	Distribution
App	80 (HTTP)	1,511,485	Benign: 71.1% Attack: 28.9%
	8080 (HTTP)		
	443 (HTTPS)		
File Server	21 (FTP)	1,394,658	Benign: 87.46% Attack: 12.54%
	22 (SFTP)		
	138 (NetBIOS)		
	139 (NetBIOS)		
	445 (SMB)		
	3389 (RDP)		
AD and Others	53 (DNS)	1,926,995	Benign: 94.42% Attack: 5.58%
	135 (RPC)		
	500 (VPN)		
	1433 (MSSQL)		
	3306 (MySQL)		
	5060 (VoIP)		

TABLE II: Final Sampled Subdataset Composition

C. Centralized Baseline

For the centralized baseline, a Graph Neural Network (GNN) architecture was adopted, specifically GraphIDS [10]. GraphIDS is a self-supervised intrusion detection model designed to learn normal network behavior directly from data without requiring labeled attacks. It operates by first using an inductive GNN to generate embeddings of each network flow, capturing its local topological context within the communication graph. These embeddings are then processed by a Transformer-based encoder-decoder structured as a masked autoencoder, which reconstructs partially hidden representations. Through this reconstruction task, the model learns both local structural patterns and global co-occurrence relationships via self-attention. Deviations from these learned patterns can subsequently be identified as potential intrusions. The architecture of the training pipeline, originally proposed by Guerra et al. [10], is shown in Figure 1.

For the centralized baseline, the training process was monitored using the following performance metrics:

- Macro F1-score
- Precision-Recall AUC
- Training time (seconds)

D. Federated Learning Setup

The centralized model was adapted into a federated setting using NVFlare, an industry-grade framework designed for scalable federated learning deployments [11]. The system consists of multiple clients simulating distributed network environments. For each job configuration, the best global model is saved and evaluated after training. Additionally, the training metrics (F1 and PR-AUC) are saved on each client for each round to later be plotted and analyzed. Lastly, the training time and data transfer are logged to evaluate the computational cost of the federated approach.

For the federated experiments, the same model architecture and training hyperparameters were used as in the centralized baseline to ensure a fair comparison. The federated training

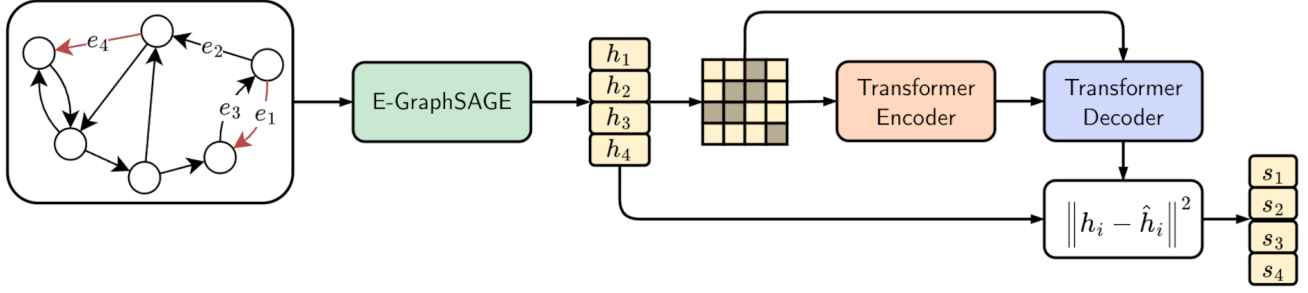


Fig. 1: Overview of the GraphIDS training pipeline (adapted from [10], Figure 3).

process was monitored using the same performance metrics as the centralized baseline, along with the additional metric of communication overhead (measured in terms of data transferred between clients and server).

To simulate a realistic federated learning scenario, NVFlare was configured to run in Proof Of Concept (POC) mode [12]. The POC command enables a proof-of-concept setup on a single machine. In this configuration, separate processes emulate the server, clients, and admin console, providing a practical environment to test and validate workflows before moving to a fully distributed deployment.

For future reference in the paper, the clients data distribution can be seen in Table III:

Client ID	Data Subsample
client0	App
client1	File Server
client2	AD and Other

TABLE III: Mapping of Client IDs to Network Traffic Subsamples

E. Secure Aggregation

F. Differential Privacy

In official papers published by NVIDIA researchers, the Opacus library was used to implement DP-SGD on each client during local training [13]. Opacus is a widely used library for integrating differential privacy into PyTorch models, providing efficient implementations of DP-SGD and tools for tracking privacy budgets.

IV. RESULTS

A. Centralized Baseline

The paper describing GraphIDS reports a macro F1-score of 0.9447 and a PR-AUC of 0.8819 on the NF-CSE-CIC-IDS2018-v3 dataset [10]. The model ran on a single NVIDIA A100 GPU with 40GB of memory, for our experiments, the model ran on a single NVIDIA RTX 4090 GPU with 24GB of memory. The obtained results were a macro F1-score of 0.9517 and a PR-AUC of 0.8939, which are slightly higher than those reported in the original paper, which can be attributed to differences in hardware.

The centralized model was trained with both the final subsample of the dataset, and a non balanced version of the dataset (AD and Others category). The results are as follows:

Metric	Final Subsample	Unbalanced Subsample
Macro F1-score	0.9205	0.9385
PR-AUC	0.6612	0.6471

TABLE IV: Model Performance Comparison Across Subsets

As shown in Table IV, the model trained on the final subsample achieved a higher PR-AUC but a lower F1-score compared to the model trained on the unbalanced subsample. This suggests that while the final subsample may have improved the model's ability to distinguish between classes (as reflected in the higher PR-AUC), it may have also introduced some noise or reduced the model's overall accuracy (as reflected in the lower F1-score). The unbalanced subsample, on the other hand, may have allowed the model to achieve a higher F1-score by leveraging the class imbalance, but at the cost of a lower PR-AUC, indicating that the model may have struggled to effectively identify positive samples in the presence of the imbalance.

B. Centralized vs Federated

C. Impact of Secure Aggregation

D. Impact of Differential Privacy

E. Trade-off Analysis

V. DISCUSSION

VI. CONCLUSION

ACKNOWLEDGMENT

The authors thank the Universidad de los Andes for all the academic support provided. To all teachers, especially Ph.D. Yuri Andrea Pinto as this thesis advisor.

REFERENCES

- [1] D. Ameria, U. Kumar, and S. Sankhe, "A comprehensive analysis of inference attacks in federated learning," in *2023 IEEE Pune Section International Conference (PuneCon)*, 2023, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/PuneCon58714.2023.10450123>

- [2] A. Khraisat, A. Alazab, S. Singh, T. Jan, and A. J. Gopez, "Survey on federated learning for intrusion detection systems: Concept, architectures, aggregation strategies, challenges, and future directions," *ACM Computing Surveys*, 2024. [Online]. Available: <https://doi.org/10.1145/3687124>
- [3] B. Yurdem, M. Kuzlu, M. K. Gullu, F. O. Catak, and M. Tabassum, "Federated learning: Overview, strategies, applications, tools and future directions," *Heliyon*, vol. 10, no. 19, p. e38137, 2024. [Online]. Available: <https://doi.org/10.1016/j.heliyon.2024.e38137>
- [4] X. He, Z. Song, D. Zhang, H. Ju, and Q. Meng, "Homomorphic encryption-based federated active learning on gcns," *Symmetry*, vol. 17, no. 6, p. 969, 2025. [Online]. Available: <https://doi.org/10.3390/sym17060969>
- [5] OWASP Foundation, "Owasp machine learning security top ten 2023," 2023. [Online]. Available: https://owasp.org/www-project-machine-learning-security-top-10/docs/ML04_2023-Membership_Inference_Attack.html
- [6] D. Torre, A. Chennamaneni, J. Jo, G. Vyas, and B. Sabrsula, "Towards enhancing privacy-preservation of a federated learning cnn intrusion detection system in iot: Method and empirical study," *ACM Transactions on Software Engineering and Methodology*, 2024. [Online]. Available: <https://doi.org/10.1145/3695998>
- [7] Canadian Institute for Cybersecurity, "A realistic cyber defense dataset (CSE-CIC-IDS2018)," Registry of Open Data on AWS, 2018. [Online]. Available: <https://registry.opendata.aws/cse-cic-ids2018>
- [8] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116. [Online]. Available: <https://doi.org/10.5220/0006639801080116>
- [9] M. Luay, S. Layeghy, N. Noorbin, M. Sarhan, G. Kulatilleke, N. Moustafa, and M. Portmann, "Time matters: Temporal NetFlow features for ML-based network intrusion detection," *IEEE Access*, vol. 14, pp. 66 899–66 913, April 2026. [Online]. Available: <https://doi.org/10.1109/ACCESS.2026.3688204>
- [10] L. Guerra, T. Chapuis, G. Duc, P. Mozharovskyi, and V.-T. Nguyen, "Self-supervised learning of graph representations for network intrusion detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.16625>
- [11] H. R. Roth *et al.*, "Empowering federated learning for massive models with nvidia flare," *arXiv preprint arXiv:2402.07792*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.07792>
- [12] NVIDIA Corporation, *NVIDIA FLARE 2.6 User Guide: Proof of Concept (POC) Command*, 2024. [Online]. Available: https://nvflare.readthedocs.io/en/2.6/user_guide/nvflare_cli/poc_command.html
- [13] H. R. Roth, S. Tickoo, M. Kumar, I. Yang, A. Liu, A. Varshney, S. Kundu, I. Vintila, P. Madsgaard, J. Milcak *et al.*, "Privacy-preserving federated fraud detection in payment transactions with NVIDIA FLARE," *arXiv preprint arXiv:2603.13617*, March 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2603.13617>